

Agent 为什么需要远端 git-rg：从代码检索到受控变更

面向开发者社区的技术文章

2026-09-30 核对：稳定版 v0.7.0；Gitee 为当前工作区中尚未发布的功能。

命令及平台边界以文档导航和各功能手册为准。

摘要

Agent 处理代码任务时，第一步往往不是修改文件，而是回答一组定位问题：符号在哪里定义？配置从哪里进入？调用链经过哪些服务？一条错误文本在哪些仓库出现？某个分支或 tag 当前究竟指向哪个 commit？

这些问题的共同点是“搜索和理解”，不是“拥有一个本地 checkout”。在小型项目里，先 `git clone` 再搜索很自然；在大型公司中，仓库数量、历史大小、并发 agent 数和凭证边界会把这个默认动作放大为基础设施问题。每个 agent 都下载 Git 历史、创建工作树、维护目录、清理缓存，然后只使用其中很小的一部分内容，成本与风险都不匹配。

`git-rg` 提供了一个更窄的边界：先把 branch、tag 或 commit 解析为不可变 commit SHA，再通过 GitHub/GitLab API（当前工作区还包括尚未发布的 Gitee 适配器）读取选定快照所需的内容，在本地用 Go RE2 匹配，并以 NDJSON 输出可编排的结果。它不创建目标仓库的 clone、checkout 或工作树，也不下载 Git 历史。

本文的核心判断很简单：对 agent 的“发现、定位、核对和跨仓库检索”阶段，远端 `git-rg` 应当是默认工具；需要少量文本变更时，可通过 `read` 和显式启用的 `propose` 提交同仓库草稿 PR/MR；需要构建、测试、`bisect`、历史分析或离线操作时，再准备本地 checkout。

1. Agent 的第一步通常是检索，而不是搬运仓库

一个典型的 agent 任务可能是：

- 找到 `TenantLifecycleGate` 的定义和所有调用点；
- 找出生产配置中某个 feature flag 的来源；
- 对照 API、worker 和部署仓库，定位一条错误码的传播路径；
- 在固定 release tag 上确认某个行为是否存在；
- 列出多个服务当前默认分支的 head commit，判断它们是否同步。

这些动作要求 agent 能够稳定地回答“在哪里”和“是哪一个版本”，而不要求它把每一个仓库的完整对象数据库和工作树搬到本地。仓库越多，检索越像一次只读查询，而不是一次开发环境初始化。

可以把 agent 的代码工作分成两个阶段：

- **发现阶段**：搜索符号、路径、配置、错误文本，确认相关仓库、ref 和 commit，读取少量上下文。

- 变更阶段：读取固定快照的完整文件，由 agent 生成文本变更并预览，必要时显式发布草稿 PR/MR；构建、测试和历史分析仍交给执行环境。

检索和受限文本变更可通过 provider API 完成；需要本地文件系统、工具链或 Git 历史的任务再进入 checkout。将每一次短查询都绑定到 clone，会让它承担开发环境的全部成本。

2. 为什么大型公司的 Clone 默认值会失真

2.1 Clone 搬运的不只是命中行

一个目标仓库的 clone 通常包含 Git 历史、对象数据库、引用、索引和工作树。即使 agent 最终只需要一行配置，冷启动阶段也要等待网络传输、对象解包、文件落盘和目录初始化。随后还要处理 checkout、并发目录隔离、清理、磁盘配额和权限。

对跨仓库任务，成本可以先用一个保守模型表示：

```
clone 成本
= 任务数 x 每任务涉及的仓库数
  x (历史传输 + 工作树落盘 + 初始化/清理 + 凭证与目录管理)
```

这不是说 clone 没有价值。它只是把“查询一个远端 snapshot”升级成了“建立一个本地开发环境”。当 agent 只需要定位文本时，升级没有带来对应收益。

2.2 企业规模会放大短生命周期浪费

大型公司常见的代码布局包括多个产品仓库、共享库、部署仓库、基础设施仓库和镜像仓库。一个问题可能只需要访问三个仓库，但 agent 的执行器需要为每个任务创建隔离目录。若任务在几分钟内结束，目录、对象和工作树仍然需要清理；若清理失败，短期峰值会变成长期的磁盘告警。

并发还会放大几个二阶问题：

- 相同 commit 被不同任务重复下载，除非平台另外维护一套可复用的对象缓存；
- 工作树包含比搜索所需更多的文件，增加本地扫描和 inode 压力；
- clone 凭证通常与 Git transport、远端 URL 或 helper 配置绑定，生命周期更长，权限面更宽；
- agent 失败、取消或超时后，半成品目录和锁文件需要回收；
- 代码检索服务很难仅凭“命中行数量”控制 clone 的网络和磁盘消耗。

2.3 一个可审计的资源边界更适合 agent

远端检索可以把请求、时间、结果和本地资源都变成显式预算。例如 `git-rg` 默认把最大匹配行数设为 200，远端 HTTP 请求上限设为 100，整体 timeout 设为 5 分钟；达到结果上限会标记结果已截断，而不是悄悄把部分结果当成完整结果。仓库实现还对单行、单文件、archive、tree、provider 响应和结果 spool 设置独立上限。

这些预算不是容量承诺，也不是把远端搜索变成零成本。它们提供的是一个清晰的失败契约：当搜索没有读完时，机器可以看到 `complete=false` 和具体 `reason`，上游 agent 可以决定是否缩小 glob、提高预算或转入 clone。

2.4 两种工作边界的对照

维度	Clone 模式	远端 git-rg
传输对象	Git 对象、引用和可选历史、工作树	固定 commit 的 tree，以及缓存、archive 或 blob 内容
版本边界	需要额外记录 checkout 状态	meta.commit 明确绑定不可变 commit SHA
生命周期	创建目录、checkout、清理和回收	单次只读查询；immutable cache 可复用
成本控制	主要由 clone 大小和并发目录决定	timeout、请求/结果/资源上限和 summary 完整性
凭证面	Git transport、远端 URL、helper 和工作区	读取使用只读身份；发布使用显式写入凭证
适合任务	本地修改、构建、测试、bisect、离线	定位、核对、只读审计，以及受控文本变更和 issue 创建

3. 远端 git-rg 的核心模型：固定快照，按需读取

3.1 先 Resolve，再读取内容

git-rg 的每一种搜索模式都会先解析 ref。输入可以是默认分支、branch、tag 或 commit SHA。各 provider 先解析仓库或 project 及 commit，最终形成一个带 commit SHA 的 snapshot。

这一顺序很重要：branch 和 tag 是会变化的名称，commit SHA 才是本次查询的不可变边界。NDJSON 的 meta event 会记录 provider、仓库、请求的 ref、解析后的 ref、commit 和 mode。agent 因此可以把结果绑定到明确版本，而不是把“当时的 main”当成隐含事实。

git-rg refs REPOSITORY 也遵守同样的只读边界。它使用 provider 的 branch/tag API，完整分页、校验、去重并稳定排序。ref 列表不写入磁盘缓存，因为 ref 会变化；head_tags 和 head_branches 只表示名称当前指向完全相同的 commit SHA，不代表历史归属或祖先关系。

3.2 完整 tree 定义范围，缓存、archive 和 blob 提供内容

固定 commit 后，exact 和 auto 先读取完整 tree，再按 glob 选择路径。内容读取按以下思路工作：

```
agent
|
v
git-rg: 解析 repository + ref
|
v
Provider API -> immutable commit SHA -> complete tree + glob
|
+---> 通过校验的缓存 / 少量 blob ----> 本地 RE2 匹配 ----> NDJSON
|
`--> archive stream -----> blob 校验 + 本地匹配
    |
    `--> 缺失、改写或 archive 不可用 ----> blob 补读
```

完整 tree 始终是搜索范围依据，不能等 archive 失败后才读取它。显式 glob 缩小范围时，可复用通过校验的 blob 缓存，并对较小的未命中文件集合直接读取 blob；较大范围通常使用 archive。archive 条目按 tree 中的 blob ID 校验，缺失或被改写的文件会补读；archive 在第一个普通文件前不可用时回退 blob。各 provider 的 REST API 细节不同，上层仍不调用 Git transport，也不创建目标仓库工作树。

这条路径减少的是 Git 历史、checkout 和 clone 生命周期，不是“只下载命中行”。对于 `exact`，程序会读取 glob 选中的普通文本文件，并在本地匹配；因此完整搜索仍可能读取所选 commit 中的全部符合条件文件。archive 也有压缩输入和解压输出上限，tree/blob fallback 也受请求、文件和资源上限约束。

3.3 三种模式必须被正确理解

模式	适合什么	完整性契约
<code>exact</code>	完整审计、确认“所选 commit 中是否存在匹配”	不调用代码索引；正常结束时 <code>complete=true</code> 。结果上限、timeout、请求/provider 错误、取消或资源上限会使结果不完整。
<code>auto</code>	日常 agent 检索，尝试先利用 provider 索引加速	默认模式。索引候选只是加速；之后仍扫描整个选定 commit。索引 warning 不会把完整 archive 扫描变成部分扫描。
<code>indexed</code>	低延迟地检查 provider 索引候选	只读索引候选，不使用 archive，也不回退全量扫描。正常结果为 <code>complete=false</code> 、 <code>reason=indexed_mode</code> ；参数或请求错误会记录对应错误原因，未命中不能证明仓库没有匹配。

`indexed` 需要模式有非空字面前缀，并受候选页数、候选路径大小等限制。Gitee 不提供公开代码索引 API，`indexed` 返回 `indexed_search_unsupported` 和退出码 2；`auto` 跳过索引加速，继续精确扫描。它可以是发现阶段的快速信号，但不应作为合规审计或“没有此符号”的证据。需要完整结论时应使用 `--mode exact`，并检查 `summary.complete`。

4. 大型企业情景模型：为什么差异会到 TB 级

下面是一个示例假设，不是行业统计，也不是 git-rg 的实测结果。它只用来展示成本如何随并发任务数放大。

假设一个企业同时运行 1,000 个 agent 任务，每个任务需要查询 3 个仓库；若采用 clone 模式，每个目标仓库本次需要传输和准备约 2 GB 的数据：

$1,000 \text{ 个任务} \times 3 \text{ 个仓库} \times 2 \text{ GB/仓库} = 6,000 \text{ GB}$ ，约 6 TB

这 6 TB 是短期冷启动传输和工作区数据的情景估算，尚未计入历史重复、重试、清理失败和对象缓存。它不表示每家公司都一定遇到这个数字。

再给远端快照一个同样明确的情景假设：每个选定 commit 的 snapshot archive 需要 300 MB，仍按每个任务、每个仓库都冷读一次计算：

$1,000 \text{ 个任务} \times 3 \text{ 个仓库} \times 300 \text{ MB/archive} = 900,000 \text{ MB}$ ，约 900 GB

900 GB 也不是 git-rg 的性能承诺；它只是一个用于比较数据搬运边界的情景数，而且未计缓存复用。真实 git-rg 的 `auto/exact` 可能读取整个选定快照，具体传输量取决于 provider、archive、glob、文件类型、请求失败和缓存命中。这个例子表达的不是“远端搜索永远比 clone 小多少”，而是：当任务只需要固定 commit 的检索时，不应默认把 Git 历史和工作树一并纳入每次任务。

更合理的生产设计通常会进一步做两件事：

- 把 commit 固定、结果完整性和资源上限写进 agent 的工具契约；
- 让相同 commit 的 immutable object 可以在受控边界内复用，而不是让每个短任务创建自己的 clone。

5. 为 agent 设计的输出、凭证和缓存契约

5.1 NDJSON 是逐事件的机器接口

搜索默认输出一行一个 JSON 对象：`meta`、若干 `match/context`，最后是 `summary`。
`warning/error` 可出现在不同阶段；认证 `warning` 可能先于 `meta`，参数校验失败可能只有 `error`。这比让 agent 解析人类终端文本更稳定：

event	agent 可用信息
meta	provider、仓库、requested/resolved ref、commit、mode；可选 commit 信息。
match	path、1-based line/column、text、每个完整命中的 submatch。
context	匹配前后文，以及 <code>before</code> 或 <code>after</code> 方向。
warning / error	稳定 code 和 message；warning 不一定使进程失败。
summary	matched/scanned 计数、缓存命中、下载字节、API 请求、transport、完整性、截断状态、reason 和耗时。

判断结果时不能只看退出码。应同时读取：

- `complete=true`：搜索按当前模式和预算正常读完；
- `complete=false`：存在截断、索引不完整、timeout、请求/provider 错误、取消或资源限制；
- `truncated=true`：达到匹配结果上限，未读后缀没有检查承诺；
- `reason`：例如 `result_limit`、`indexed_mode`、`request_budget`、`cancelled`、`resource_limit` 或 `transport` 错误原因。

5.2 Cache 复用不可变对象，不缓存会变化的 refs

默认启用持久化 immutable object cache，容量总计为 512 MiB。它按 commit 固定的 tree、blob、archive 和 indexed raw-file 条目保存内容，每个 `v2.entry` 内保存版本头、SHA-256 校验和与正文，以一次 rename 发布完整条目。旧正文与 sidecar 格式不再复用。token 不参与缓存 key；branch/tag 列表不落盘缓存，因为它们是可变的。

`--no-cache` 会禁用本次运行的持久化磁盘缓存读写和清理。它不会禁用运行时临时结果 `pool`；`match/context` 事件仍可能先写入受保护的 OS 临时文件。对高敏感任务，agent 应同时隔离 `stdout`、`stderr`、日志、产物和 OS 临时目录，因为搜索结果可能包含私有源码、路径和 `commit message`。

5.3 只读 token 让权限与查询边界一致

GitHub 场景优先使用只授权目标仓库、`Contents: read` 的 fine-grained token；GitLab 场景优先使用受限 `project/group access token` 或 `PAT`，并按实例要求使用 `read_api`。不要为只读搜索工具授予 `api` 或 `write_repository`。读取可使用环境变量或默认 `--auth auto` 复用匹配站点的 `gh/glab` 登录；`--auth env` 关闭 CLI 读取。Gitee 仅使用环境变量凭证。token 不能放进仓库 URL，也没有 token CLI flag。

这和 `clone` 的权限面不同：读取命令只调用 provider API 获取快照及元数据，凭证可由 agent 运行时注入。自 v0.6.0 起，没有适用的读取环境变量时，会先回退到 `GITRG_WRITE_TOKEN`，再考虑 CLI 登录；需要独立读取身份时显式设置 `GITRG_TOKEN`。写入始终只使用 `GITRG_WRITE_TOKEN`，不会由读取凭证替代。无论使用何种模式，输出和日志仍须按敏感数据保护，不能把“只读”误解为“内容不敏感”。

6. 三个可运行的查询示例

下面的命令来自当前 README 的 CLI 契约。示例中的仓库、token 和模式都应替换为组织自己的值。

示例一：对 GitHub 私有仓库做完整 Go 文件审计

```
export GITRG_TOKEN='read-only-token'
git-rg --mode exact \
  --glob '*.go' \
  --glob '!vendor/**' \
  --context 2 \
  --max-results 200 \
  --max-requests 100 \
  --timeout 2m \
  -F 'TenantLifecycleGate' \
  github:ACME/control-plane
```

agent 读取最后的 `summary`，只有在 `complete=true` 时才把“完整审计没有更多结果”作为结论。若达到 200 行，`complete=false`、`truncated=true`、`reason=result_limit`。

示例二：在不 clone 的情况下查看 branch/tag 当前 head

```
git-rg refs --kind all --format ndjson \
  github:ACME/control-plane
```

这个命令返回 `branch/tag` 的当前 `commit`。相同 SHA 的名称可以出现在关联字段中，但该结果不表示历史分支关系，也不下载 `commit` 历史。

示例三：对自建 GitLab 的固定 release 做远端检索

```
export GITRG_TOKEN='read-only-token'
git-rg --provider gitlab \
  --api-base https://gitlab.example.com/api/v4 \
  --ref v2.4.0 \
  --mode exact \
  --format ndjson \
  -F 'retry_after' \
  git@gitlab.example.com:platform/edge/router.git
```

SSH 风格地址在这里仅用于解析仓库地址；`git-rg` 不使用 SSH 认证或 Git transport。provider 会把 `v2.4.0` 解析为固定 commit，再通过 API 读取该 snapshot。

7. 不 clone 的受控变更与 issue 工作流

搜索片段不应直接当作完整文件。自 v0.5.0 起，agent 可以用 `read` 获取固定 commit 的完整 UTF-8 文本及 blob SHA，生成包含原始 blob 身份的 JSON 计划，再预览或发布：

```
git-rg read --ref main github:ACME/control-plane path/to/file.go
git-rg propose --dry-run --changes changes.json github:ACME/control-plane
git-rg propose --enable-write --changes changes.json github:ACME/control-plane
```

`changes.json` 中包含 `base_commit`、目标分支、新源分支、标题、commit message 和 `create/update/delete` 动作；`update/delete` 必须携带 `read` 返回的 `expected_blob`。后续文件读取用同一个 commit 固定快照。完整格式见 Agent changes (`agent-changes.md`)。

写入默认关闭。每次发布都需要 `--enable-write` 和具备相应平台权限的 `GITRG_WRITE_TOKEN`；`--dry-run` 优先于写入开关，仅读取远端。发布会检查基础分支和文件冲突，在同仓库新分支创建一个 commit 和草稿 PR/MR，并核验完整 tree。`result.complete=true` 只表示远端提交及 PR/MR 已确认，不表示编译、测试或 CI 通过。

自 v0.7.0 起还可创建 issue；其预览完全在本地，不解析 commit，也不需要凭证：

```
git-rg issue create --dry-run --title "Investigate regression" \
  --body-file issue.md github:ACME/control-plane
git-rg issue create --enable-write --title "Investigate regression" \
  --body-file issue.md --agent-name "Review Agent" \
  --agent-run-id "run-123" github:ACME/control-plane
```

Agent 名称、模型和 run ID 是自报身份，不改变平台作者，也不是 issue 幂等键。提案响应丢失时，用相同计划、仓库/API base 和身份恢复；issue 响应不确定时，先检查远端列表，再决定是否重试。两者都不自动重放写请求。详见 Issue creation (`issues.md`)。

GitLab 使用 `gitlab:GROUP/PROJECT`；包含未发布 Gitee 适配器的构建使用 `gitee:OWNER/REPO`。Gitee issue 的字符串编号放在 `result.identifier`，GitHub/GitLab 保留数字 `result.number`。不能把本地 Gitee 功能当成 v0.7.0 安装包已支持的能力。

8. 什么时候用 git-rg，什么时候需要本地 checkout

可以把选择规则压缩成一张清单：

优先使用远端 git-rg

- 只需要定位符号、配置、错误文本或路径；
- 需要跨多个仓库做发现和比较；
- 需要把结果绑定到某个 branch、tag 或 commit 的当前 snapshot；
- 需要可机器消费的 NDJSON、完整性和资源统计；
- 只读身份可以访问 provider API，且任务不需要离线操作；
- 要预览少量普通文本变更、提交同仓库草稿 PR/MR，或创建 issue，并能显式提供所需写入权限。

需要本地执行环境、clone 或 checkout

- 需要已有工作树、未提交文件、复杂本地编辑或通用 Git 操作；
- 要运行编译、测试、lint、生成器或 IDE 工具；
- 要执行 git bisect、历史搜索、祖先关系分析或提交级别审阅；
- 要依赖本地路径、子模块、Git LFS、构建缓存或未被远端 API 暴露的工具链；
- 要在没有网络的环境中反复读取和操作同一份代码。

因此，“远端 git-rg”不是“永远不要 clone”。它是对 agent 工作流的分层：把检索和可核验的受限 API 变更留在远端，把需要本地状态和工具链的工程操作交给执行环境。

9. 结论：把查询和变更都放进可验证的边界

大型公司的代码库不是一个可以无限复制的目录，而是一组需要按权限、版本和资源边界访问的事实。对于多数 agent 任务，先 clone 会让 agent 为了得到几行文本，承担历史传输、工作树落盘、清理和凭证管理的全部成本。

远端 git-rg 给出了更接近任务本质的接口：

- 通过 provider API 把可变 ref 解析为不可变 commit；
- 先以完整 tree 定义范围，再利用缓存、archive 和 blob 读取并核验内容；
- 用 auto、exact、indexed 明确速度和完整性取舍；
- 用 NDJSON 的 meta/match/context/summary 让 agent 能检查版本与完整性；
- 用 request、timeout、result、resource limits 和 512 MiB immutable cache 控制成本；
- 读取使用最小权限身份，写入要求显式开关与独立写入凭证；
- 通过完整文件和 blob 身份生成变更，分别检查搜索完整性、远端发布结果和 CI。

真正重要的不是“有没有 clone”，而是工具是否把任务需要的状态缩小到可验证的最小边界。对发现阶段而言，这个边界就是远端固定 commit 上的 rg 查询；受限文本变更可通过显式启用的 API 写入完成；工具链执行和历史分析再进入本地 checkout。

依据与范围说明

本文依据 2026-09-30 的 v0.7.0 发布状态及当前工作区源码核对。Gitee 属于尚未发布的工作区改动，不引用外部统计，也不把示例数字写成行业测量。主要依据包括：

- [README.md](#) 与 [README.zh-CN.md](#) : 无 clone 语义、仓库地址、搜索模式、NDJSON、refs、权限、缓存和资源边界；
- [internal/provider/git_repository.go](#)、[internal/provider/github.go](#)、[internal/provider/gitlab.go](#)、[internal/provider/gitee.go](#) : [ref/commit resolve](#)、[tree/blob/archive](#) 和 [refs](#) 分页；
- [internal/search/runner.go](#)、[internal/search/archive.go](#) : 完整 tree、[exact/auto/indexed](#) 语义、[archive/blob](#) 校验和完整性处理；
- [internal/search/types.go](#)、[internal/output/output.go](#) : NDJSON event 和 summary 字段；
- [internal/cache/cache.go](#)、[internal/cli/cli.go](#) : 默认 512 MiB immutable cache 和 [--no-cache](#) 行为；
- [internal/cli/changes.go](#)、[internal/proposal/](#)、[internal/cli/issues.go](#) : 完整文件读取、变更计划、写入开关、Agent 身份、提案恢复与 issue 不确定响应。

产品边界也必须保留：v0.7.0 支持 GitHub/GitLab，当前工作区新增尚未发布的 Gitee；没有离线模式、本地路径搜索、历史搜索或通用 Git server 协议。文本变更仅支持同仓库新分支，不更新已有分支，不处理二进制、LFS、symlink、submodule 或自动合并，也不执行仓库代码。[exact/auto](#) 的完整扫描仍可能读取整个选定 commit，[indexed](#) 永远是不完整模式。任何生产接入都应在自己的 provider、权限、仓库和资源预算上做验收。